

WorkGraph POC 四篇 Paper High-Level Reading 與 Roadmap

日期：260427

脈絡：Vulkan persistent-thread / software WorkGraph producer-consumer POC

0. 為什麼讀這四篇

我們現在的 Vulkan POC 已經不只是「一個 compute shader 產生 Koch snowflake」。它其實是一個小型 GPU-side runtime：

- worker 透過一次 compute dispatch 長駐在 GPU 上；
- task 透過 software queue 傳遞；
- queue state、payload visibility、ready flag 依賴 atomic；
- producer / consumer 比例會影響 spin-wait、global memory traffic、cache locality；
- termination detection 也是 scheduling system 的一部分。

這四篇 paper 可以串成一條技術演化路線：

```
persistent global queue
-> batched / hierarchical queue
    -> wavefront / staged queues
        -> continuation / coroutine-style runtime
```

我們短期仍然應該停在「Vulkan compute 可以手刻」的範圍，但後面兩篇可以幫我們看見更大的架構方向。

1. 四條技術路線總覽

技術路線	代表 paper	核心理念	對我們 POC 的意義
Persistent threads + software queues	Cederman and Tsigas 2008	GPU worker 長駐，透過 shared queue 動態取 task	這就是目前 baseline。先量出 global queue + atomics 到底在哪裡爆。
Specialized / batched concurrent queues	Troendle, Ta, and Jang 2019	用 group-level proxy / batch queue operation 降低 contention 和 retry	下一個最實際的優化方向：把 per-task atomic 改成 group-level reservation。
Wavefront / staged execution	Laine, Karras, and Aila 2013	不把所有邏輯塞在一個 megakernel，而是拆成 coherent stages	用來比較：single persistent shader 是否應該拆成 stage-aware queues 或多個 stage。
GPU coroutines / continuation runtime	Zheng et al. 2024	用 coroutine/state frame/continuation 把手動拆 stage 的工作自動化	長期架構參考：task payload 可以變成 continuation token + state frame。

2. Paper 1: On Dynamic Load Balancing on Graphics Processors

來源：Daniel Cederman and Philippas Tsigas, "On Dynamic Load Balancing on Graphics Processors", Graphics Hardware 2008.

連結：<https://diglib.eg.org/items/80970ff5-d565-4a2e-88cc-cc9701e4ad9e>

這篇在解什麼問題

這篇問的是一個很根本的問題：

如果 GPU workload 是動態的、每個 task 的成本事先不知道，那能不能像 CPU 一樣做 dynamic load balancing？

答案是可以，但 synchronization 非常貴。

它比較了多種 GPU dynamic load balancing 方法，包含 lock-based 和 lock-free 設計，並用 octree construction 這類不規則 workload 做實驗。對我們來說，重點不是照抄 2008 年的演算法，而是它很早就指出：

- dynamic GPU work 需要 scheduling；
- queue / shared state 會成為核心成本；
- lock-free 比 blocking 更適合 GPU，但 lock-free 不等於便宜；
- synchronization 必須被當成一個要量測的主角。

對我們的啟發

這篇基本上就是 persistent-thread scheduling 的歷史 baseline：

CPU dispatch 一批 persistent workers
workers 在 GPU 上一直拿 task
task 可以產生 child task
shared queue / counter 透過 atomic 同步

這和我們目前 POC 很像：

- workgroups 是 persistent workers；
- q1 / q2 是 software queues；
- Task.payload[5] 是 ready flag；
- queue count / head / tail 透過 atomic 管；
- NodeB / NodeC 角色目前由 workgroup 分配。

所以這篇 paper 對我們的定位是：

目前版本不是錯，它是必要 baseline；但它會暴露出 global queue + atomic scheduling 的天然瓶頸。

我們應該量什麼

從這篇出發，baseline 不應該只量 FPS。至少要補：

- q1 / q2 enqueue count ;
- q1 / q2 dequeue count ;
- CAS success / failure ;
- full-queue retry ;
- empty-queue retry ;
- ready-flag spin 次數 ;
- NodeB / NodeC 每個 workgroup 處理的 task 數 ;
- stop flag / termination 觸發時機。

可能會看到的問題

預期 baseline 會在這些地方卡住：

- 很多 lanes 同時做 per-task atomic ;
- producer 比 consumer 快，queue 壓力升高；
- consumer 預約到 slot 但 payload 還沒 ready，只能 spin；
- payload 都經過 global memory，locality 差；
- queue empty 不代表整個 graph 真的完成，termination detection 變複雜。

這些不是失敗，而是我們後續設計的理由。

3. Paper 2: A Specialized Concurrent Queue for Scheduling Irregular Workloads on GPUs

來源：David Troendle, Tuan Ta, and Byunghyun Jang, "A Specialized Concurrent Queue for Scheduling Irregular Workloads on GPUs", ICPP 2019.

連結：<https://par.nsf.gov/servlets/purl/10188451>

這篇在解什麼問題

這篇直接瞄準 persistent-thread scheduler 的痛點：

active threads 越多，shared queue 上的 contention 和 retry 越嚴重。

它提出一個 GPU 專用的 non-blocking concurrent queue。對我們最重要的是兩個概念：

1. queue operation 不會因為 CAS 失敗而反覆 retry；
2. 一次 queue operation 可以處理多筆 entries，成本接近處理一筆；
3. 每個 thread group 用一個 proxy thread 代表整個 group 執行 atomic operation。

這等於把問題從：

每個 lane / 每個 task 都去搶 global queue atomic

改成：

一個 proxy lane / workgroup 先保留一批 slots
group 內部用 local offset 分配
payload 連續寫入

對我們的啟發

我們現在的 enqueue 大概是：

每產生一個 task:
CAS / atomic 更新 queue count
atomic tail reserve slot
write payload
atomic publish ready flag

這很容易讓 atomic 次數和 task 數量成正比。

這篇 paper 告訴我們：GPU scheduler 的 queue 不該讓每個 lane 都直接碰 global queue metadata。應該把 atomic 粒度提高到 group level。

可以手刻的 POC variant

第一個可以做的是 batch enqueue：

NodeB workgroup:
先計算這個 workgroup 產生多少 child tasks
proxy lane atomic reserve N 個 queue slots
lanes 根據 local offset 寫 payload
最後 publish 這批 tasks

第二個可以做的是 NodeC output batch allocation：

Before:
每個 leaf edge atomicAdd vertexCount, 2

After:
每個 workgroup 先統計 leafCount
atomicAdd vertexCount, 2 * leafCount
group 內部用 local offset 寫 vertex buffer

第三個可以做的是 local queue + global spill：

workgroup LDS queue:

先在 local queue 裡傳 task

local queue 滿了才 spill 到 global queue

我們應該量什麼

- 每個 output edge 平均幾次 atomic ;
- CAS failure 是否下降 ;
- q1 / q2 full retry 是否下降 ;
- batch size 對 frame time 的影響 ;
- batch 後 payload write 是否更連續 ;
- proxy lane 是否造成 group 內部 idle 。

這條路的價值

這是短期最值得做的優化路線。原因很樸素：

- 仍然是 Vulkan compute 可手刻；
- 不需要改整個架構；
- 可以直接對應現在最可疑的 atomic contention；
- 結果容易量化。

但它不一定解決 payload locality。queue atomic 變少，不代表 payload 不走 global memory。這就是為什麼後面還需要 hierarchical queue 或 staged execution。

4. Paper 3: Megakernels Considered Harmful

來源：Samuli Laine, Tero Karras, and Timo Aila, "Megakernels Considered Harmful: Wavefront Path Tracing on GPUs", High-Performance Graphics 2013.

連結：https://research.nvidia.com/publication/2013-07_megakernels-considered-harmful-wavefront-path-tracing-gpus

這篇在解什麼問題

這篇來自 rendering / path tracing 領域。它的觀點是：

把整個複雜 renderer 塞進一個巨大 GPU kernel，通常不是好主意。

原因包括：

- control flow divergence 嚴重；
- 某些 lanes 被 mask 掉，SIMT utilization 差；
- register usage 太高，occupancy 降低；
- instruction cache 壓力高；
- 不同 material / ray state 混在一起，coherence 差。

所以它主張 wavefront path tracing：把大 kernel 拆成多個比較單純的 stage，用 queue 串起來。

例如不要這樣：

```
one megakernel:
  ray generation
  intersection
  material evaluation
  shadow ray
  next bounce
  output
```

而是這樣：

```
ray queue
intersection queue
material queue
shadow queue
next-bounce queue
output queue
```

它的 tradeoff 是：

```
多一些 queue traffic / stage transition
換取更好的 coherence、較低 register pressure、較高 occupancy
```

對我們的啟發

我們現在的 shader 還不算真正的 megakernel，但已經有小型 megakernel 的形狀：

```
NodeA seed logic
NodeB subdivision logic
NodeC output logic
queue logic
termination logic
```

如果未來 Node 變多、payload 變複雜、branch 變多，單一 persistent shader 可能會遇到：

- NodeB / NodeC 行為差異太大；
- register pressure 被最重的 node 拉高；
- 角色分配固定，某些 workgroup 空等；
- branch divergence 變重。

Wavefront 思路提供另一種架構對照：

不一定要讓一個 persistent shader 內部處理所有 node；也可以把 node/stage 拆開，讓每一段更 coherent。

可以手刻的 POC variant

我們可以設計幾個架構對照：

Variant A: current single persistent shader

NodeA / NodeB / NodeC 都在同一個 shader

Variant B: stage-aware persistent shader

仍然一次 dispatch，但 worker 根據 queue pressure 選 NodeB 或 NodeC

Variant C: split compute shaders

NodeB 和 NodeC 拆成不同 shader / pass

Variant D: staged queues + batch reservation

stage queue 仍存在，但 queue operation 採用 batch reservation

我們應該量什麼

- shader register count；
- occupancy；
- NodeB / NodeC branch divergence；
- 每個 stage 的 GPU time；
- stage queue traffic；
- split shader 是否減少 spin-wait；
- split shader 是否增加 global memory traffic。

這條路的價值與風險

價值：

- 可以測 single persistent shader 是否真的適合 WorkGraph-like workload；
- 可以讓 NodeB / NodeC 分別 tuning；
- 可能降低 register pressure 和 divergence。

風險：

- 多 stage 可能增加 queue traffic；
- 如果用 CPU multi-dispatch，會偏離 GPU self-scheduling；
- 如果 payload locality 是主因，staging 反而可能更慢。

所以這條路不是「一定會快」，而是必要的 architecture comparison。

5. Paper 4: GPU Coroutines for Flexible Splitting and Scheduling of Rendering Tasks

來源：Shaokun Zheng, Xin Chen, Zhong Shi, Ling-Qi Yan, and Kun Xu, "GPU Coroutines for Flexible Splitting and Scheduling of Rendering Tasks", ACM Transactions on Graphics 43(6), Article 281, 2024.
連結：<https://cg.cs.tsinghua.edu.cn/people/~kun/2024/GPUCoroutines.pdf>

這篇在解什麼問題

這篇可以看成 wavefront 的更高層版本。

Wavefront 很有用，但手動拆 stage 很痛苦：

- programmer 要決定在哪裡 split；
- 每個 stage 要保存哪些 state 很難管理；
- nested control flow 很難拆；
- queue / continuation / state frame 都要手寫；
- scheduler 和 payload layout 會綁死在 application 上。

GPU coroutine 的想法是：

```
programmer 用接近 megakernel 的方式寫程式
在需要切開的地方標記 suspend point
compiler/runtime 自動抽出 subroutine
runtime materialize coroutine state frame
scheduler 負責恢復 continuation
```

概念上：

```
megakernel-like code + suspend marks
-> compiler extracts continuation subroutines
-> runtime stores coroutine state frames
-> scheduler resumes the right subroutine later
```

對我們的啟發

我們短期不應該真的做一個 GPU coroutine compiler。那太大。

但這篇提供一個很好的抽象：

```
task = continuation token + state frame
node execution = resume from a suspension point
scheduler = 決定下一個 resume 哪個 continuation
payload = continuation 需要保存的 live state
```

這和 Work Graph 的精神很接近。

可以轉成我們的手刻方向

目前 task 是：

```
struct Task {
    uint payload[6];
};
```

未來可以變成 continuation-style task：

```
struct Task {
    uint nodeId;           // resume at NodeB / NodeC / future node
    uint stateOffset;      // offset into state-frame buffer
    uint depth;
    uint flags;
};
```

shader 內部變成：

```
switch (task.nodeId) {
case NODE_B:
    run_subdivide(task);
    enqueue_continuation(NODE_B or NODE_C, newState);
    break;

case NODE_C:
    run_output(task);
    break;
}
```

這樣 queue entry 不一定要攜帶完整 payload，而是可以只帶：

```
nodeId + stateOffset + scheduling metadata
```

payload / state frame 放在另一個 buffer。

我們應該量什麼

- queue entry size 是否下降；
- payload bytes per generated task 是否下降；
- state-frame indirection 是否增加 memory latency；
- continuation-style task 是否讓 irregular graph 更容易表達；
- scheduler 是否更容易做 dynamic role / stage selection。

這條路的價值與風險

價值：

- 更接近 Work Graph / runtime abstraction ；
- node 增加時不需要重寫所有 queue layout ；
- payload 可以壓縮成 state pointer / token ；
- 方便探索 dispatcher / locator 。

風險：

- state-frame lifetime 管理變難 ；
- 多一層 indirection ；
- debug 難度增加 ；
- termination detection 更複雜 。

所以這是中長期架構，不是第一個 optimization 。

6. 四篇 Paper 的共同結論

6.1 先讓 dynamic work 跑起來

Paper 1 告訴我們：persistent workers + software queues 可以處理 dynamic workload，但 synchronization 是核心成本。

對應行動：

- 保留目前 POC 當 baseline ；
- 把 queue / atomic / spin / termination 量出來 。

6.2 再讓 queue operation scale

Paper 2 告訴我們：per-lane / per-task atomic 不會 scale。queue 要為 GPU SIMT 執行模型特化。

對應行動：

- batch reservation ；
- proxy lane / proxy workgroup ；
- group-level output allocation ；
- LDS local queue + global spill 。

6.3 再考慮 execution coherence

Paper 3 告訴我們：如果一個 shader 裡有太多不同 stage，可能應該拆開。

對應行動：

- 比較 single persistent shader 和 stage-aware queues ；
- 測 split NodeB / NodeC 是否降低 register pressure 或 divergence ；
- 評估 queue traffic 是否抵消好處 。

6.4 最後走向 continuation/runtime abstraction

Paper 4 告訴我們：長期不應該永遠手寫 queue protocol 和 state passing。

對應行動：

- 把 task 視為 continuation token；
- 把 payload 視為 state frame；
- 讓 dispatcher / locator 變成可替換 scheduler policy。

7. 建議 Roadmap

Phase 0: Freeze Koch Baseline

目的：建立穩定對照組。

工作：

- 固定 `MAX_DEPTH`、`QUEUE_SIZE`、`NODE_C_START`；
- 確認 vertex count / output correctness；
- 記錄目前 compute time / frame path；
- 保存一份 baseline config。

產出：

- baseline 設定表；
- baseline metric table；
- correctness 截圖或驗證紀錄。

Phase 1: Runtime Metrics

目的：讓 scheduler 成本可見。

加入：

- GPU timestamp：reset / compute / barrier / render；
- q1 / q2 enqueue count；
- q1 / q2 dequeue count；
- CAS success / failure；
- full queue retry；
- empty queue retry；
- ready flag spin count；
- max queue occupancy；
- NodeB / NodeC processed task count；
- output allocation count；
- termination trigger timing。

產出：

- 每次 run 的 CSV / console table ;
- profiler capture ;
- bottleneck note 。

Phase 2: Queue Optimization Variants

目的：直接驗證 Paper 2 。

Variants :

1. current global queue baseline ;
2. batch enqueue for q1 / q2 ;
3. group-level output allocation ;
4. batch dequeue where practical ;
5. local LDS queue + global spill 。

要回答：

- atomic count 是否下降？
- CAS failure 是否下降？
- queue occupancy 是否更平滑？
- bottleneck 是 atomic 還是 payload traffic？

Phase 3: Workload Expansion

目的：避免只對 Koch 最佳化。

Workloads :

1. Koch baseline：規則、固定 expansion；
2. irregular expansion：每個 task 產生 0 / 1 / 2 / 4 / N 個 child；
3. producer-heavy imbalance：NodeB 產生 work 比 NodeC 快；
4. consumer-heavy imbalance：NodeC 經常等 NodeB；
5. output-heavy leaf workload：特別壓 vertex allocation / memory write 。

要回答：

- optimization 是否只對規則 workload 有效？
- queue pressure burst 時會發生什麼？
- NodeB / NodeC 是否需要 dynamic role switching？
- output write 是否是 hidden bottleneck？

Phase 4: Stage-Aware / Wavefront Architecture

目的：驗證 Paper 3 的架構替代路線。

Variants :

1. current single persistent shader；
2. stage-aware persistent shader；

3. split NodeB / NodeC compute shaders ;
4. staged queues + batch reservation 。

要回答：

- 拆 stage 是否降低 register pressure ?
- occupancy 是否提高 ?
- stage queue traffic 是否變成新瓶頸 ?
- WorkGraph-like workload 更適合 persistent runtime 還是 wavefront runtime ?

Phase 5: Continuation-Style Runtime

目的：借用 Paper 4 的 coroutine 模型，但不做完整 compiler 。

設計：

- queue entry 包含 `nodeId`、`stateOffset`、scheduling metadata ;
- payload / state frame 放到獨立 buffer ;
- NodeB / NodeC 成為 resume point ;
- dispatcher 決定下一個 resume 哪個 continuation 。

要回答：

- token-based queue 是否減少 payload traffic ?
- state-frame indirection 是否太貴 ?
- continuation model 是否讓 irregular graph 更容易擴展 ?
- 這是否能成為 Vulkan software WorkGraph 和 native WorkGraph 之間的概念橋樑 ?

8. 實驗 Backlog：從 Paper 轉成可執行 Variant

這裡的原則是：

每個 variant 只驗證一個假設。

不要一次把 queue batching、LDS local queue、dynamic role switching、continuation token 全部混在一起。那樣就算效能變好，也很難說清楚到底是哪個設計有效。

優先級	Variant	來源 paper	要驗證的假設	主要 metrics	預期產出
P0	Current Koch baseline + metrics	Paper 1	現在的 global queue runtime 成本可被拆解	GPU timestamp、CAS failure、spin count、queue occupancy	baseline table
P1	Group-level output allocation	Paper 2	NodeC output 不應該每個 leaf 都做 global atomic	output atomic count、vertex write time、NodeC time	output bottleneck report
P1	Batch enqueue for q1 / q2	Paper 2	per-task enqueue atomic 是主要 contention 來源之一	enqueue atomic count、CAS failure、queue retry	batching comparison
P2	Irregular expansion workload	Paper 1 + Paper 2	Koch 太規則，無法代表 bursty dynamic workload	max queue occupancy、full retry、role idle time	workload stress report
P2	Producer/consumer imbalance workload	Paper 1 + Paper 3	固定 NodeB / NodeC 比例可能無法處理 queue pressure	q1/q2 occupancy、NodeB/NodeC processed count、idle/spin	role allocation note
P3	Stage-aware persistent shader	Paper 3	worker 應該根據 queue pressure 選 stage，而不是固定角色	processed count per role、empty retry、compute time	stage-aware comparison
P3	Split NodeB / NodeC compute shaders	Paper 3	拆 stage 可能降低 register pressure / divergence	occupancy、register count、stage time、queue traffic	wavefront architecture comparison
P4	LDS local queue + global spill	Paper 2	producer-consumer locality 可以減少 global queue traffic	global enqueue count、spill count、payload bytes	locality experiment
P4	Continuation-style task format	Paper 4	queue entry 可以從 full payload 變成 nodeld + stateOffset	queue entry bytes、state-frame load cost、extensibility	continuation design note

建議第一輪實作順序

第一輪不要碰太大架構，先做能快速驗證的三件事：

1. **baseline + metrics**：先讓問題可見。
2. **group-level output allocation**：範圍最窄，最容易驗證是否有改善。
3. **batch enqueue for q1/q2**：直接測 paper 2 的主要假設。

第一輪完成後，才進入 workload expansion：

```
Koch baseline
-> irregular expansion
-> producer-heavy imbalance
-> consumer-heavy imbalance
-> output-heavy leaf workload
```

這樣可以避免只把系統調到「剛好適合 Koch」。

Variant 評估格式

每個 variant 建議都用同一個小表：

欄位	說明
Hypothesis	這次只驗證哪一個假設
Code change	改了哪個 shader / C++ path
Correctness	vertex count / output 是否仍正確
Metrics delta	相對 baseline 的 timestamp / atomic / retry / occupancy 變化
Interpretation	這個結果支持或否定哪個 scheduler 設計
Next variant	下一步應該接哪個實驗

這個格式會讓每次優化都能回到研究問題，而不是變成單純 trial-and-error。

9. 建議研究 framing

可以把研究題目暫時寫成：

We study hand-crafted GPU-side scheduling techniques for approximating Work Graph-style execution in Vulkan, moving from persistent global queues toward batched, hierarchical, staged, and continuation-based runtimes.

中文說法：

我們研究如何在 Vulkan 上手刻 GPU-side scheduler 逼近 Work Graph-style execution；路線從 persistent global queue 出發，逐步探索 batched queue、hierarchical queue、wavefront staging，以及 continuation-style runtime。

這個 framing 的好處是：

- 不是只說「我們做了一個比較快的 shader」；
- 而是說明 software WorkGraph runtime 的成本組成；
- 再說哪些手刻技術可以降低哪些成本；

- 最後指出哪些剩餘成本需要 native WorkGraph / runtime / API support。

10. 立即下一步

建議順序：

1. 對目前 Koch baseline 加 metrics。
2. 實作 group-level output allocation，因為範圍窄、最容易驗證。
3. 實作 q1 / q2 batch enqueue。
4. 加 irregular expansion workload。
5. 比較 fixed NodeB/NodeC role 和 simple dynamic role switching。
6. 再考慮 LDS local queue 或 continuation-style task。

原則：

一次只驗證一個 hypothesis。

不要第一輪就把 batching、local queue、dynamic role switching、continuation payload 全部混在一起。那樣就算跑快了，也很難解釋為什麼。

11. References

- Daniel Cederman and Philippas Tsigas. "On Dynamic Load Balancing on Graphics Processors." Graphics Hardware 2008. <https://diglib.eg.org/items/80970ff5-d565-4a2e-88cc-cc9701e4ad9e>
- David Troendle, Tuan Ta, and Byunghyun Jang. "A Specialized Concurrent Queue for Scheduling Irregular Workloads on GPUs." ICPP 2019. <https://par.nsf.gov/servlets/purl/10188451>
- Samuli Laine, Tero Karras, and Timo Aila. "Megakernels Considered Harmful: Wavefront Path Tracing on GPUs." High-Performance Graphics 2013. https://research.nvidia.com/publication/2013-07_megakernels-considered-harmful-wavefront-path-tracing-gpus
- Shaokun Zheng, Xin Chen, Zhong Shi, Ling-Qi Yan, and Kun Xu. "GPU Coroutines for Flexible Splitting and Scheduling of Rendering Tasks." ACM Transactions on Graphics 43(6), Article 281, 2024. <https://cg.cs.tsinghua.edu.cn/people/~kun/2024/GPUCoroutines.pdf>